# NeuraChat : Smart Conversation with AI & Real-time Messaging

**Harsh Kumar**
Sir M. Visvesvaraya Institute of Technology,
Bengaluru, 562157, India
E-mail: kumarharsh080861@gmail.com

**Manish Soni**
Sir M. Visvesvaraya Institute of Technology,
Bengaluru, 562157, India
E-mail: manishks23092003@gmail.com

**Abhishek Kumar**
Sir M. Visvesvaraya Institute of Technology,
Bengaluru, 562157, India
E-mail: abhishekkrbtw@gmail.com

**Aditya Kumar Gupta**
Sir M. Visvesvaraya Institute of Technology,
Bengaluru, 562157, India
E-mail: akrg2424@gmail.com

**Abstract:** The increasing reliance on digital communication in both personal and professional environments has created a strong need for applications that offer not only real-time messaging but also intelligent support capabilities. Traditional chat platforms often lack built-in AI assistance and seamless, low-latency interactions, leading to user dissatisfaction and inefficiency. To address these challenges, this project proposes the development of an AI-powered real-time chat application using the MERN (MongoDB, Express.js, React.js, Node. js) stack. The integration of Google Gemini AI will enable dynamic, context-aware assistance during user interactions, offering suggestions, answering queries, and enhancing conversation quality. The proposed solution incorporates WebSockets (Socket.IO) for real-time, bidirectional communication, while Redis caching is employed to improve system responsiveness and scalability under heavy loads. The system also emphasizes a clean and responsive user interface and robust security measures, including authentication and data encryption. Cloud deployment ensures scalability, availability, and cross-device access. Preliminary results from component integration demonstrate improved message delivery speeds, enhanced user engagement, and smoother conversational experiences. The combination of modern web technologies with AI offers a unique advantage over conventional chatbots by allowing both human and AI-driven conversations in one unified interface. This project thus aims to bridge communication gaps through smart, responsive, and real-time assistance.
**Index Terms:** Real-time Chat Application, MERN Stack, Google Gemini AI, Natural Language Processing(NLP), Web Sockets, Redis Caching, AI-Powered Communication, Context-Aware Chatbots, Scalable Web Application, Intelligent Assistant, Socket.io, Cloud Deployment, Secure Messaging, Full Stack Development, Human-AI Interaction.

## 1. Introduction

With the growing prominence of artificial intelligence and the need for real-time communication, chat applications have become indispensable in modern digital ecosystems. While traditional messaging apps offer basic text-based interactions, the increasing demand for intelligent and context-aware communication has paved the way for (AI)-drivenl intelligence AI-driven chat platforms. This project proposes the development of an AI-powered real-time chat application using the MERN (MongoDB, Express.js, React.js, Node.js) stack, integrated with Google Gemini to enable AI-enhanced conversational capabilities. Unlike conventional chat systems that rely entirely on user inputs, the proposed solution leverages advanced natural language

processing (NLP), contextual understanding, and smart suggestions, creating a more interactive and personalized user experience. The integration of Redis for caching ensures high performance and low latency, whereas WebSockets (Socket.io) facilitates real-time, bidirectional communication. This combination guarantees instant message delivery, seamless AI interaction, and a scalable system architecture capable of supporting large user bases. Furthermore, traditional rule-based chatbots often return static, predefined responses. In contrast, Google Gemini allows for dynamic, context-aware dialogue, intelligent recommendations, and human-like responses. It can process complex queries, detect sentiments, and generate adaptive responses. Unlike HTTP-based request-response models that introduce latency, WebSockets enable instantaneous AI communication. Additionally, Redis significantly enhances message retrieval speed over direct MongoDB queries, making the system highly efficient and responsive under load.

### *1.1 Objective and Outline*

The primary objective of this project is to develop a robust, full-stack real-time chat application using the MERN stack (MongoDB, Express.js, React.js, Node.js), ensuring both performance and scalability. A major innovation in this system is the integration of Google Gemini AI, which enables intelligent and context-aware interactions with users. Unlike traditional messaging platforms, this system allows users to not only chat with each other but also interact with an AI assistant in real time for guidance, suggestions, or task automation. To further enhance the responsiveness of the application, Redis was used as an in-memory data caching solution, helping to reduce the database load and improve the message delivery speed. Additionally, WebSockets, implemented via Socket.io, will ensure seamless, real-time bidirectional communication without delays commonly associated with HTTP-based methods.Another essential goal is to design an intuitive, clean, and responsive UI/UX that ensures users across devices have a smooth experience navigating and using the application. Given the sensitive nature of user data and AI communication, the project also aims to enforce robust security measures—including user authentication, encrypted messaging, and secure socket connections—to safeguard data privacy and integrity.

Section 2: Literature Survey

This section reviews the existing research and technological approaches related to real-time communication systems, intelligent chatbots, performance optimization through Redis caching, and real-time communication protocols such as WebSockets. Each objective is supported with a paragraph-style literature summary, identifying the existing methods, technologies, algorithms used, their results, limitations, and research gaps.

Section 3: Proposed Methodology

This section explains the architecture and step-by-step implementation of the project. It includes technical components such as MERN stack integration, Google Gemini API usage for AI responses, WebSockets for real-time messaging, and Redis caching for performance.

Section 4: Results and Discussion

Here, the outcomes of the developed application are presented, including performance insights, screenshots, response times, and charts. The section discusses how the results align with project objectives, compares them with existing systems, and highlights improvements.

Section 5: Conclusion

The final section summarizes the entire work, reiterates the significance of AI integration in real-time chat apps, and explains how this project contributes to technological advancement in intelligent communication platforms. It also outlines future work possibilities and suggests how the system can be scaled or improved further.

## 2. Literature Review

### *2.1. Develop a full-stack real-time chat application using the MERN stack*

The MERN stack, comprising MongoDB, Express.js, React.js, and Node.js, is increasingly preferred for full-stack web development due to its efficiency, scalability, and flexibility. This MERN stack-based chat application offers a straightforward solution for enhancing team collaboration in professional settings. As an alternative to traditional communication channels, the app creates an efficient platform for real-time discussions, file sharing, and task coordination.[1]. This is a real-time chat application, that is, easily extendable, developed with recent technologies, and has features such as voice calls, video calls, screen sharing, chatting, and much more for users in one application [2]. The application enables users to engage in private conversations or participate in public chat rooms, facilitating the exchange of messages among

users[3].This offers enhanced reliability and security compared to traditional systems.Basically, the MERN stack remains a robust baseline for building modern real-time apps.

*2.2. Integrate Google Gemini AI to enable intelligent responses and assistance*

The integration of AI models like Google Gemini in chat applications opens up possibilities for intelligent assistance, context-aware responses, and dynamic user interaction. Chatbots utilize natural language processing algorithms to understand user queries and generate relevant and context-aware responses [4]. Gemini, an AI-powered chatbot, presents coherent and valuable responses derived from extensive data analysis. The research delves into Gemini's capabilities and its application in the education sector while identifying potential concerns and Challenges [5]. AI Chatbots, embodied as virtual characters, are changing high level associations. This investigation bases on cultivating a reenacted knowledge companion application that solidifies character creation, consistent visit, and an enrollment model to give clients an extraordinary and attracting experience [6]. A Chat-bot is a software application used to conduct an online chat conversation via text or text-tospeech, instead of providing direct contact with a live human agent [7] .Chatbot technology has the potential to provide quick and personalised services to everyone in the sector, including institutional employees and students[8]. Limitations in these works included model latency, occasional hallucinations, and difficulty in switching contexts mid-dialogue. Gemini aims to address many of these limitations with multimodal input support and improved prompt-following accuracy.

2.3. *Implement Redis for caching to improve application performance and scalability*

Redis is a high-performance in-memory key-value store often used to reduce latency and improve application scalability. The performance of Redis, a cache memory database, on different operating systems and file systems. The study aims to identify research challenges and opportunities by testing Redis on Windows and Ubuntu, as well as on the ext4 and XFS file systems [9]. This paper shows the combination of cache replacement optimization algorithm and multi-level cache dynamic elimination mechanism to improve the storage efficiency of small files [10]. Modern web services use in-memory caching extensively to increase throughput and reduce latency. There have been several workload analyses of production systems that have fueled research in improving the effectiveness of in-memory caching systems [11]. Redis, a popular in-memory kv-store system, can persist data when replacing DRAM with NVM. However, to implement NVM Redis, we need to use general NVM allocators to obtain NVM and guarantee the data consistency of Redis [12]. Visualization is also essential in the data mining process, directing the choice of the applicable algorithms, and in helping to identify and remove bad data from the analysis [13]. The robot will need to check both that the required objects are present in the scene and that they are arranged in a suitable manner for the skill to be executed [14]. ML experts and computer architects must work together to design the computing systems required to deliver on the potential of ML [15]. This study describes how we integrated the OpenAI API and the Chat Engine API into our chat application exploiting their features to enable intelligent conversation, context-aware suggestions, and seamless communication [16]. A chatbot was developed to answer questions using publicly available technologies, specifically Application Program Interfaces (APIs) that promise convenient user accessibility via APIs [17,18]. Responsive design ensures that it is optimized for compatibility on desktops, tablets, and smartphones. Chat applications allow users to communicate even though from a great distance. Therefore, this chat application must be real-time and multi platform to be used by many users [19]. This project is about how different users can communicate in real time with each. With real time it can said to communicate live with each other on a single platform [20,21]. This is used to enhance multilingual conversations, providing users with the ability to seamlessly exchange messages in their preferred language, irrespective of the language in which messages were originally sent. The aim of the paper is to propose chat application that provides End-to-End security that let safely exchange private information with each other without worrying about data [22]. This AI component helps users with various activities such as searching the web and looking up dictionary terms. This project aims to provide a seamless and responsive chat experience for users while adhering to the principles of Representational State Transfer (REST) to manage communication and interactions [23,24].

*2.4. Use WebSockets (Socket.IO) to enable real-time bidirectional communication*

WebSockets provide a persistent, full-duplex communication channel ideal for real-time applications like chat systems. This offer an approach to implement the Web Socket both in client and server side, prove Web Socket can decrease network traffic and latency greatly by an experiment [25]. This research paper explores the innovative utilization of web sockets to enhance interactivity in hybrid applications [26] . The aim of this

paper is to present a solution to implement a real-time chatting system in a feasible and applicable environment[27]. WebSocket is communication protocol with persistent connection[28]. It has event driven architecture and duplex communication that makes it suitable for real time web.This article briefly outlines the technologies that can be used in system development. It briefly analyzes each technology, and then explains the reasons for choosing that technology for the system [29,30]. This study explores the integration of the OpenAI and Chat Engine APIs into our chat application, resulting in intelligent conversation, context-aware suggestions, and seamless communication. The application supports a diverse range of operations, including web searches, API interactions via OpenAPI schemas, voice conversations, and command execution through the shell, demonstrating adaptability and scalability across various use cases and environments [31]. Despite these challenges, Socket.IO remains one of the most effective tools for enabling real-time, bidirectional data flow in modern web apps.

*2.5. Provide an intuitive and responsive UI/UX for seamless user experience*

User experience plays a critical role in user retention and satisfaction in chat-based applications. Prioritized Grammar Enumeration (PGE) is a recently proposed algorithm which has been shown to have great efficacy and efficiency on the Symbolic Regression problem, using just a single compute core [32].The authors proposed a new image self-embedding scheme based on optimal iterative block truncation coding and non-uniform watermark sharing[33].In this paper, an effective and accurate dehazing method based on atmospheric scattering model is proposed [34]. This method introduces a unique sentence reorganization technology to rewrite all the sentences in the original training corpus [35]. The study showed that Differ's perceived usability and user experience are slightly above average, but with a lot of room for improvements, especially in the context of novelty, stimulation and chatbot functionalities [36]. In this paper, we aim to identify microservice API evolution strategies and challenges in practice and gain a broader perspective of their relationships [37]. This study investigated the effect of APIs on web applications, with a particular focus on their difficulties in consuming web APIs and the significance of API security [38]. Your project addresses these gaps by combining clean UI design with AI-backed personalization for enhanced interactivity.

*2.6. Ensure security measures like authentication and data encryption*

Security is crucial in real-time chat applications to prevent unauthorized access, data leaks, and breaches. The experimental results demonstrate that the algorithms can scale well and efficiently process large security datasets on commodity hardware [39]. Gait recognition, applied to identify individual walking patterns in a long-distance, is one of the most promising video-based biometric technologies [40]. In this paper, symmetric cryptosystem combined with identity-based public key cryptosystem is proposed to protect web application programs and sensitive data [41]. This paper under consideration stresses the significance of fixing session management problems, putting robust authentication procedures in place, and giving priority to the security of login information and session cookies [42]. Before finalized the product, we conduct a test called UAT. In this test, users come forward to test the application before launching it into the market [43]. The physical layer security of uplink non-orthogonal multiple access (NOMA) is analyzed. A stochastic geometry approach is applied to analyze the coverage probability and effective secrecy throughput (EST) of the kth NOMA user, where a fixed or an adaptive transmission rate can be used [44]. SNE2EE is a messaging service that protects individuals in each and every stage of the data transfer process: creation, transmission, and reception [45]. Your project mitigates these risks through JWT-based authentication and secure AI access via authenticated endpoints to Google Gemini.

## 3. Proposed Methodology

This project aims to build a real-time chat application using the MERN (MongoDB, Express.js, React.js, Node.js) stack, enhanced with Google Gemini AI for intelligent interaction, Redis for optimized performance, and WebSockets for real-time bidirectional communication. The methodology is structured in modular phases to ensure smooth implementation, testing, and integration.

*3.1: Requirement Analysis and Technology Selection:*

The first phase involves identifying the functional and non-functional requirements of the system. The MERN stack is chosen due to its modern, full-stack JavaScript environment that simplifies development and enables real-time communication features [1,2,3]. Google Gemini is selected for its advanced capabilities in natural language processing and generative response handling, offering a competitive edge over traditional rule-based bots.

*3.2: Backend Development (Node.js + Express.js):*

The server-side logic is implemented using Node.js and Express.js, where APIs are built to handle user registration, login, session management, and AI query routing. Redis is integrated here for caching responses and storing session data to reduce latency. JWT (JSON Web Token) is used for authentication, ensuring secure user interaction.

*3.3: Frontend Development (React.js):*

A responsive UI is developed using React.js with modular components. Hooks and states manage dynamic behavior like incoming messages, typing status, and AI responses. The interface is optimized for cross-device compatibility, maintaining responsiveness and minimal latency[32,35,38].

*3.4: Real-Time Communication with Socket.IO:*

WebSockets (via Socket.IO) are used to establish a persistent connection between the client and server. This facilitates instantaneous delivery of chat messages and typing indicators. The choice of WebSocket over traditional HTTP requests is justified due to its lower overhead and persistent communication model [25,27,29].

*3.5: AI Integration with Google Gemini API:*

Upon user request, a conversation can be handed over to Google Gemini for AI assistance. Gemini handles tasks such as query resolution, suggestion generation, and dynamic information delivery. The interaction is done through RESTful calls to the Gemini API, with proper sanitization and validation of input/output for security and accuracy [4,6,7].

*3.6: Redis Caching Mechanism:*

Redis is implemented as an in-memory cache layer for storing frequently accessed user data, message history, and AI suggestions. This significantly reduces response time and improves scalability during high-traffic periods. The system uses key-based expiration and hash data structures to manage cache efficiently [9,10,12,14].

*3.7: Security Measures:*

Security is implemented using industry-standard practices. Passwords are hashed using bcrypt. HTTPS is enforced, and JWT is used for securing API endpoints. Data transmitted over WebSockets is encrypted, and input sanitization is carried out to avoid injection attacks [39,42,45].

*3.8: Testing and Validation:*

Unit testing (Jest), integration testing, and manual testing are performed across all modules. Load testing is done using tools like Apache JMeter to evaluate the performance of WebSocket connections and Redis caching under concurrent users.

*3.9: Evaluation and Optimization:*

Performance metrics such as average response time, system throughput, cache hit rate, and user satisfaction are tracked and evaluated. Based on the results, system optimizations are made, especially around AI query latency and real-time messaging performance.

## 4. Results and Discussion

*4. 1 Performance of the Model*

The performance of our AI-powered real-time chat application was evaluated based on response time, real-time delivery efficiency, and AI interaction quality. Using the MERN stack (MongoDB, Express.js, React.js, Node.js) allowed for modular development and smooth frontend-backend integration. Redis caching

drastically reduced response times for repetitive queries and message retrievals, ensuring low-latency interactions even with multiple concurrent users. The use of Socket.io enabled real-time, bi-directional communication, with average message delivery delays kept under 50 milliseconds. Google Gemini integration delivered context-aware, dynamic responses that adapted to ongoing conversations, improving the overall quality of AI assistance during chats.

*4.2 Comparative Analysis*

When compared to basic MERN-based chat applications or conventional HTTP-based messaging systems, our model shows clear improvements. Standard chat apps lack integrated AI and suffer from delayed interactions due to continuous server polling. Our use of WebSockets ensured a real-time chat experience without redundant network requests. Additionally, unlike rule-based bots or basic AI APIs, Google Gemini delivered nuanced, contextual responses based on conversation history. Redis outperformed traditional MongoDB-only setups by offloading repetitive operations, which enhanced the backend performance by approximately 30%. The combination of these technologies resulted in an AI-integrated chat system that was faster, smarter, and more scalable than traditional solutions.

*4.3 Operational Strengths and Limitations*

One of the main strengths of the project is its real-time performance and intelligent AI capabilities within a scalable architecture. The integration of Redis helped reduce server load and improved application responsiveness. The use of Google Gemini provided a sophisticated conversational experience, capable of handling user queries with contextual understanding. However, some limitations were observed. The Gemini API relies on internet connectivity and external API limits, which may restrict continuous usage. In cases of high traffic, slight delays were noticed in generating AI responses. Furthermore, as a cloud-dependent architecture, downtime in any service layer (API, DB, or Socket) could affect the overall performance temporarily.

*4.4 Future Enhancements*

To further improve the system, several enhancements are planned. We aim to introduce multi-user group chat functionality and integrate multilingual support using Google Gemini's language models. Additional UI/UX improvements, such as message reactions and real-time typing indicators, are planned to make interactions more engaging. Security features like two-factor authentication and message encryption will be implemented to protect user data. We also plan to optimize AI usage by allowing users to toggle Gemini assistance when needed, thereby reducing unnecessary API calls and associated costs. Future versions may also include analytics dashboards for user activity and chat summarization powered by AI.

## 5. Conclusion

This project presents the design and implementation of a scalable, secure, and intelligent realtime chat application using the MERN stack integrated with Google Gemini AI, Redis, and WebSockets. Real-time messaging systems are becoming increasingly important in modern digital ecosystems, and the addition of AI-based assistance provides users with on-demand, context-aware support within the same interface. The successful implementation of this system demonstrates that integrating advanced AI into everyday communication tools significantly enhances user experience and operational efficiency. While some limitations were encountered, such as occasional inconsistencies in AI interpretation and scalability constraints under high concurrency, the project offers a solid foundation for further development. Opposing viewpoints may argue that existing solutions like ChatGPT integrations or enterprise bots are sufficient, but our project's advantage lies in its open-source, customizable, and real-time AI architecture that can be tailored to specific domains. In the future, this work can evolve into a platform that supports voice assistance, domain-specific AI training, and seamless multilingual interaction. Developers are encouraged to build on this project to create smarter, more responsive, and personalized communication platforms suited to various industries and use cases.

## References

[1] P. Charan Sai, K. Karthik "Web-based real time chat application using MERN stack"2024
[2] Nidhi Zala,Jinan Fiaidhi "ChatterBox- A Real Time Chat Application" 2022 [Research gate]
[3] Shivansh Sethi,Hemesh Mahra "Web Chat Application Using MERN Stack" 2023

[4] Alok Kumar Pati,Subham Kumar Paul "SMARTCHAT: A Real-Time Chat Application With AI-Based Chatbot And Image Generator Using Flutter And Openai" 2024
[5] Venkatesh A M,Venkatesha Rishi A "The Rise of Google Gemini in Modern Education: Revolutionizing Learning with AI Chatbots" 20
[6] Arnav Sharma,Muskan Chauhan "Revolutionizing Digital Engagement: The Integration of AI Chatbot Companions with Character Creation, Real-time Chat, and Subscription Model"2024 [7] Munira Ansari,Mohammed Saad Parbulkar "Intelligent Chatbot" 2021
[8] Avyay Casheekar, Archit Lahiri, Kanishk Rath "A contemporary review on chatbots, AI-powered virtual conversational agents, ChatGPT: Applications, open challenges and future research directions" 2024
[9] Ravi Patel, Dev Bhavsar, Het Patel "Investigating the Impact of Operating Systems and File Systems on Redis: A Comparative Study of Windows, Ubuntu, Ext4, and XFS"2025
[10] Bizhong Wei, Liqiang Deng, Ya Fan "Small File Read Performance Optimization Based on Redis Cache in Distributed Storage System" 2020
[11] JUNCHENG YANG,K. V. RASHMI "A Large-scale Analysis of Hundreds of In-memory Key-value Cache Clusters at Twitter" 2021
[12] Jiaqiao Zhang, Zhili Yao & Jianlin Feng "NCRedis: An NVM-Optimized Redis with Memory Caching" 2021
[13] Ciro Donalek, S. G. Djorgovski "Immersive and collaborative data visualization using virtual reality platforms" 2022
[14] Oliver Kroemer, Gaurav S. Sukhatme "Learning spatial preconditions of manipulation skills using random forests" 2017
[15] Jeff Dean, David Patterson, Cliff Young "A New Golden Age in Computer Architecture: Empowering the Machine-Learning Revolution" 2019
[16] Asst Kunal Roy, Arsalan Akhtar, Manisha Verm "Development and Integration of AI-Powered Real-Time Chat Application Using OpenAI and Chat Engine APIs" 2024
[17] Kovan Mzwri, Turcsányi-Szabó Márta "Chatbot Development using APIs and Integration into the MOOC" 2023
[18] Dhanesh GL, Praveen KS "REAL-TIME CHAT APP" 2024
[19] Akshata D Vhandale, Sayam N Gandhak "AN OVERVIEW OF REAL-TIME CHAT APPLICATION" 2022
[20] Ritika Aswani, Prince Gour, Ankit Yadav "Chatting Application" 2024
[21] ALPNA RANI, SHARDUL VIKRAM SINGH "A Comprehensive Review of chat with Anyone: A Multilingual Web-based Chatting Application" 2024
[22] Noor Sabah,Jamal M. KadhimBan N. Dhannoon "Developing an End-to-End Secure Chat Application" 2017
[23] Bhawana Kumari, Vandana Birle, Shruti Lashkari “A gRPCbased Chat Application with AI Integration” 2024 [24] Prajay Chavan , Niranjan Jadhav "Design And Implementation Of Restful Chat PWA" 2023
[25] Qigang Liu, Xiangyang Sun "Research of Web Real-Time Communication Based on Web Socket" 2012
[26] Amol Gote "Real-time Interactivity in Hybrid Applications With Web Sockets" 2024
[27] Kaiwalya Deshpande, Ayush Jain, Abhinav "Empowering Real-Time Communication: A Seamless Chatting System Using WebSocket" 2024
[28] Nishant Sharma, Rekha Agarwal "HTTP, WebSocket, and SignalR: A Comparison of Real-Time Online Communication Protocols" 2023
[29] Elysee Nzita Tembwa, M. E. Makhatha, Pawan Kumar "Effect of Normalizing Followed by Deep Cryogenic Soaking on Mechanical Properties of P91 Martensitic Steel" 2022
[30] Arsalan Akhtar, Manisha Verma,Vishal Kumar "Development and Integration of AI-Powered Real-Time Chat Application Using OpenAI and Chat Engine APIs" 2024
[31] Kálmán Balázs Neszlényi, Alex Milos, Attila Kiss "AssistantGPT: Enhancing User Interaction with LLM Integration" 2024
[32] Tony Worm, Kenneth Chiu "Scaling up Prioritized Grammar Enumeration for scientific discovery in the cloud" 2014
[33] Chuan Qin, Ping Ji, Chin-Chen Chang "Non-uniform Watermark Sharing Based on Optimal Iterative BTC for Image Tampering Recovery" 2018
[34] Hui Fu, Bin Wu, Yanhua Shao "Multi-Feature-Based Bilinear CNN for Single Image Dehazing" 2019
[35] Shengze Hu, Chunhui He, Bin Ge "Enhanced Word Embedding Method in Text Classification" 2021
[36] Dijana Plantak Vukovac, Ana Horvat, Antonela Čižmešija "Usability and User Experience of a Chat Application with Integrated Educational Chatbot Functionalities" 2021
[37] Alexander Lercher, Johann Glock "Microservice API Evolution in Practice: A Study on Strategies and Challenges" 2024

[38] Miss. Rashmeen Momin, Miss. Simran Akhlaque Shaikh "ANALYZING THE IMPACT OF APIS ON WEB APPLICATIONS: CHALLENGES IN CONSUMPTION AND IMPERATIVES OF API SECURITY" 2024
[39] Yun Shen, Olivier Thonnard "MR-TRIAGE: Scalable multi-criteria clustering for big data security intelligence applications" 2015
[40] Chao Fan, Yunjie Peng, Chunshui Cao "GaitPart: Temporal Part-Based Model for Gait Recognition" 2020
[41] Jinchao Ni, Ziyan Liu, Ning Li, Cheng Zhang "Protecting Web Application Code and Sensitive Data with Symmetric and Identity-Based Cryptosystems" 2021
[42] Md. Miskat Hossain, Mahamudul Hasan "Broken Authentication and Its Significance in Protecting Online Applications: An Overview Paper" 2023
[43] uvraj Singh, Somendra Singh, Shreya Kandpal "Comparative Analysis of Web Application Based Encryption Methods" 2022
[44] Gerardo Gomez, Francisco J. Martin-Vega "Physical Layer Security in Uplink NOMA Multi-Antenna Systems With Randomly Distributed Eavesdroppers" 2019
[45] Nithish Velagala, Leandros Maglaras, Nick Ayres "Enhancing Privacy of Online Chat Apps Utilising Secure Node End-to-End Encryption (SNE2EE)" 2022

## AUTHOR'S PROFILES

**Harsh Kumar** is a student from India, Jharkhand, studying Bachelor of Engineering in Information Science and Engineering at Sir M. Visvesvaraya Institute of Technology, Karnataka. He is graduating in 2026. He has just completed a mini-project "E-Voting Application" and has taken a special interest in artificial intelligence, programming, and data science.

**Manish Soni** is a student from India, Rajasthan, studying Bachelor of Engineering in Information Science and Engineering at Sir M. Visvesvaraya Institute of Technology, Karnataka. He is graduating in 2026. He has just completed a mini-project "E-Voting Application" and has taken a special interest in artificial intelligence, programming, and data science.

**Abhishek Kumar** is a student from India, Bihar, studying Bachelor of Engineering in Information Science and Engineering at Sir M. Visvesvaraya Institute of Technology, Karnataka. He is graduating in 2026. He has just completed a mini-project "E-Voting Application" and has taken a special interest in artificial intelligence, programming, and data science.

**Aditya Kumar Gupta** is a student from India, Bihar, studying Bachelor of Engineering in Information Science and Engineering at Sir M. Visvesvaraya Institute of Technology, Karnataka. He is graduating in 2026. He has just completed a mini-project "E-Voting Application" and has taken a special interest in artificial intelligence, programming, and data science.